

Fascicule du TP1

12 février 2026

1 Analyse de la Configuration du Système

La première capture d'écran présente les paramètres de l'hyperviseur **Oracle VirtualBox** pour la machine virtuelle dédiée au TP.

1.1 Caractéristiques Matérielles (Provisionnement)

Pour assurer la stabilité des simulations NS-3, les ressources suivantes ont été allouées :

- **Système** : La mémoire vive (RAM) est fixée à **6854 Mo**, ce qui permet une compilation fluide via l'outil Ninja. L'ordre d'amorçage privilégie l'optique puis le disque dur.
- **Processeur** : L'allocation de **3 cœurs** permet d'accélérer les temps de calcul et de compilation de la bibliothèque NS-3.42.
- **Accélération** : La paravirtualisation est configurée en mode **KVM**, optimisé pour les invités Linux.

1.2 Stockage et Affichage

- **Contrôleur IDE** : Le lecteur optique contient l'image `VBoxGuestAdditions.iso`. Ces "Additions Invité" sont cruciales pour rendre l'interface **responsive** et permettre le mode plein écran.
- **Contrôleur SATA** : Le disque dur virtuel (`ubuntuVM.vdi`) dispose d'une taille de **25,04 Go**, formaté en stockage dynamique.

2 Étude du Script de Simulation : first.cc

La seconde capture d'écran montre le code source C++ du premier exemple de simulation réseau. Ce script définit une topologie simple.

2.1 Topologie et Bibliothèques

Le script commence par l'inclusion des modules fondamentaux de NS-3 :

```
1 #include "ns3/core-module.h"
2 #include "ns3/network-module.h"
3 #include "ns3/internet-module.h"
4 #include "ns3/point-to-point-module.h"
5 #include "ns3/applications-module.h"
```

- **Core** : Gère le moteur de simulation et la planification des événements.
- **Network** : Définit les nœuds (*Nodes*) et les paquets (*Packets*).
- **Internet** : Implémente la pile TCP/IP, incluant les adresses IP et le routage.
- **Point-to-Point** : Configure la couche de liaison pour une connexion physique directe entre deux nœuds.
- **Applications** : Contient les modèles de trafic (Echo Client/Server).

2.2 Initialisation du Log

La ligne suivante permet de définir un composant de journalisation (*logging*) :

```
1 NS_LOG_COMPONENT_DEFINE ("FirstScriptExample");
```

Ce composant permet d'afficher les messages de débogage dans la console lors de l'exécution pour vérifier le bon échange des paquets.

3 Conclusion du TP1

La mise en place de la machine virtuelle avec des ressources CPU/RAM suffisantes constitue la base matérielle. La compréhension des modules C++ (**Point-to-Point** et **Internet**) est l'étape logicielle indispensable pour simuler des échanges de données de manière réaliste.

Analyse des directives d'inclusion (#include)

Les lignes de code observées permettent d'importer les modules nécessaires à la construction de la simulation. Chaque fichier d'en-tête (*header*) correspond à une couche ou une fonctionnalité spécifique de NS-3 :

- **ns3/core-module.h** : C'est le cœur du simulateur. Il gère le moteur de simulation, la planification des événements (*Scheduler*), le temps système et les outils de journalisation (*Logging*).
- **ns3/network-module.h** : Ce module définit les objets de base du réseau. Il permet de manipuler les nœuds (*Nodes*), les paquets de données (*Packets*) et les interfaces réseaux.

- `ns3/internet-module.h` : Il est responsable de la pile protocolaire TCP/IP. Il inclut la gestion des adresses IP (IPv4/IPv6), le routage, et les protocoles de transport comme UDP et TCP.
- `ns3/point-to-point-module.h` : Ce module est spécifique à la couche physique et liaison. Il permet de créer des canaux de communication directs entre deux nœuds, en définissant le débit (*DataRate*) et le délai de propagation (*Delay*).
- `ns3/applications-module.h` : Il contient les modèles d'applications qui génèrent du trafic. Dans le cas du script `first.cc`, il permet d'utiliser les classes `UdpEchoServer` et `UdpEchoClient` pour simuler un échange de requêtes et de réponses.

4 Analyse des Directives et Modules (FirstscriptExample)

Le script `first.cc` introduit des fonctionnalités avancées par rapport aux exemples précédents, notamment la gestion des fichiers externes.

4.1 La bibliothèque de flux de fichiers

```
1 #include <fstream>
```

Contrairement aux modules `ns3`, `fstream` appartient à la bibliothèque standard C++. Son inclusion est nécessaire pour :

- **Écriture** : Sauvegarder les statistiques de simulation (débit, délai) dans un fichier `.dat` ou `.txt`.
- **Lecture** : Charger des scénarios de mobilité ou des paramètres réseau depuis un fichier externe.
- **Analyse** : Générer des fichiers de traces personnalisés exploitables par des outils comme Gnuplot ou Excel.

Exemple utilisant `fstream` Analyse de la syntaxe `std::ofstream`

Dans NS-3, l'extraction de données vers un fichier externe repose sur la bibliothèque `<fstream>`. Voici le détail de la syntaxe utilisée pour générer des fichiers de résultats.

4.2 Décomposition de la commande d'ouverture

```
1 std::ofstream out ("congestion-window.dat", std::ios::out);
```

- `std::` : Indique que nous utilisons l'espace de noms de la bibliothèque standard C++.
- `ofstream` : Signifie *Output File Stream* (flux de sortie de fichier). C'est la classe spécialisée pour **écrire** des données.
- `out` : C'est le nom de l'objet (ou variable) que nous créons. Nous l'utiliserons ensuite pour envoyer des données vers le disque.
- `"congestion-window.dat"` : Le premier paramètre est le nom du fichier qui sera créé ou écrasé à la racine du dossier de simulation.
- `std::ios::out` : C'est le mode d'ouverture. Il précise au système que le fichier doit être ouvert en mode **écriture**.

4.3 Écriture et Flux de données

```
1 out << simulator::Now().GetSeconds() << " " << newV << std::endl;
```

- `out <<` : L'opérateur de flux `<<` redirige la donnée vers le fichier plutôt que vers la console.
- `simulator::Now().GetSeconds()` : Récupère le temps exact de la simulation au moment de l'événement.
- `" "` : Insère un espace entre le temps et la valeur. C'est crucial pour que le fichier soit lisible par des logiciels de tracé comme **Gnuplot** ou **Excel** (format de colonnes).
- `std::endl` : Insère un saut de ligne et force l'écriture sur le disque (*flush*), garantissant que les données ne restent pas bloquées dans la mémoire tampon si la simulation s'arrête brutalement.

4.4 Fermeture du flux

```
1 out.close();
```

Cette commande libère le fichier. Si vous oubliez `close()`, le fichier risque d'être corrompu ou vide à la fin de la simulation.

4.5 Inclusions des modules et Espace de noms

- `"tutorial-app.h"` : Indique l'utilisation d'une classe d'application personnalisée définie dans le tutoriel, permettant de simuler des comportements de trafic plus complexes qu'un simple Echo.
- `using namespace ns3;` : Cette directive permet d'appeler les fonctions du simulateur (ex : `NodeContainer`, `PointToPointHelper`) sans préfixer chaque commande par `ns3::`, simplifiant ainsi la lecture du code.

4.6 Composant de Journalisation (Logging)

```
1 NS_LOG_COMPONENT_DEFINE("FifthScriptExample");
```

Cette macro est fondamentale pour le débogage. Elle définit un identifiant unique pour ce script. En utilisant la variable d'environnement `NS_LOG`, l'utilisateur peut activer des messages de trace spécifiques à ce fichier dans le terminal pour suivre l'exécution de la simulation en temps réel.

5 Analyse de la Topologie et du Point d'Entrée (main)

Le script commence par définir la structure physique du réseau et initialise les composants logiciels nécessaires à l'exécution de la simulation.

5.1 Topologie réseau par défaut

Le schéma en commentaire décrit une topologie **Point-to-Point** (P2P), qui est la forme la plus simple de réseau :

- **Nœuds (Nodes)** : Deux entités de calcul, nommées `n0` et `n1`.

- **Liaison Physique** : Une ligne directe reliant les deux nœuds. Dans NS-3, cela implique la création d'un `PointToPointChannel`.
- **Sous-réseau IP** : Les interfaces réseau de ces nœuds appartiennent au réseau 10.1.1.0, qui est une adresse de classe A privée couramment utilisée pour les tests.

5.2 L'espace de noms ns3

```
1 using namespace ns3;
```

Cette directive est essentielle pour la lisibilité. Elle indique au compilateur d'inclure l'espace de noms global de NS-3. Sans cette ligne, chaque objet devrait être appelé par son nom complet (ex : `ns3::NodeContainer` au lieu de `NodeContainer`).

5.3 Définition du composant de log

```
1 NS_LOG_COMPONENT_DEFINE("FirstScriptExample");
```

Cette macro enregistre le script auprès du système de journalisation de NS-3. Elle permet à l'utilisateur d'activer des messages de suivi spécifiques via le terminal avec la commande : `export NS_LOG=FirstScriptExample=level_all`.

5.4 La fonction main

```
1 int main(int argc, char* argv[])
```

Il s'agit du point d'entrée du programme.

- `argc` (argument count) : Le nombre d'arguments passés en ligne de commande.
- `argv` (argument vector) : Un tableau de chaînes de caractères contenant les arguments.

Dans NS-3, ces paramètres sont souvent transmis à la classe `CommandLine` pour permettre de modifier les variables de simulation (comme le nombre de paquets ou le débit) sans avoir à recompiler le code

6 Analyse détaillée du script de simulation

Cette section détaille les étapes de configuration du réseau, de la gestion des arguments à l'installation des applications.

6.1 Configuration initiale et Logging

- `CommandLine cmd(__FILE__);` : Initialise l'analyseur de ligne de commande. Cela permet de modifier les variables de simulation au moment du lancement sans modifier le code source.
- `cmd.Parse(argc, argv);` : Analyse les arguments passés par l'utilisateur dans le terminal.
- `Time::SetResolution(Time::NS);` : Définit la résolution temporelle minimale de la simulation. Ici, elle est fixée à la **nanoseconde**, ce qui est la précision standard pour les événements réseau.

- `LogComponentEnable(...)` : Active les messages d'information (`LOG_LEVEL_INFO`) pour les applications Echo Client et Server. Cela permet de voir les paquets envoyés et reçus dans le terminal.

6.2 Création des Nœuds et Topologie Physique

- `NodeContainer nodes; nodes.Create(2);` : Crée deux objets « Nœud » vides qui représentent les ordinateurs ou routeurs de la simulation.
- `PointToPointHelper pointToPoint;` : Utilise une classe d'aide pour configurer une liaison point-à-point.
- `SetDeviceAttribute("DataRate", ...)` : Définit le débit binaire de la carte réseau (ici **5 Mbps**).
- `SetChannelAttribute("Delay", ...)` : Définit le délai de propagation du signal dans le câble (ici **2 ms**).
- `NetDeviceContainer devices = pointToPoint.Install(nodes);` : Installe les cartes réseau (*NetDevices*) sur les nœuds et les connecte via le canal physique.

6.3 Pile Internet et Adressage IP

- `InternetStackHelper stack; stack.Install(nodes);` : Installe la pile de protocoles (TCP, UDP, IP) sur chaque nœud.
- `Ipv4AddressHelper address;` : Prépare l'attribution des adresses IP.
- `address.SetBase("10.1.1.0", "255.255.255.0");` : Définit le réseau (10.1.1.0) et le masque de sous-réseau (255.255.255.0).
- `address.Assign(devices);` : Attribue physiquement une adresse IP unique à chaque interface réseau des nœuds.

6.4 Configuration des Applications (Serveur et Client)

- `UdpEchoServerHelper echoServer(9);` : Prépare un serveur UDP sur le port **9**.
- `echoServer.Install(nodes.Get(1));` : Installe le serveur sur le deuxième nœud (*n1*).
- `serverApps.Start(Seconds(1.0));` : Le serveur commence à écouter à partir de 1 seconde de simulation.
- `UdpEchoClientHelper echoClient(interfaces.GetAddress(1), 9);` : Configure le client pour envoyer des données vers l'adresse IP du serveur sur le port 9.
- `SetAttribute(...)` : Définit les paramètres du client : 1 seul paquet, un intervalle d'une seconde, et une taille de 1024 octets.
- `clientApps.Start(Seconds(2.0));` : Le client commence l'envoi à 2 secondes de simulation, laissant le temps au serveur de s'initialiser.

6.5 Lancement et Fin de la Simulation

Cette dernière partie du code gère le cycle de vie de l'exécution et le nettoyage des ressources mémoire.

- `clientApps.Stop(Seconds(10.0));` : Définit la fin de l'activité du client à 10 secondes. Il est important de noter que l'arrêt de l'application n'arrête pas forcément la simulation, mais seulement l'envoi de paquets.

- `Simulator::Run()` ; : C'est la commande la plus critique. Elle cède le contrôle au moteur d'événements de NS-3. Le simulateur commence alors à traiter la file d'attente des événements planifiés (Start server, Start client, envoi de paquets, etc.) dans l'ordre chronologique. Le programme reste bloqué sur cette ligne jusqu'à ce qu'il n'y ait plus d'événements ou qu'une condition d'arrêt soit rencontrée.
- `Simulator::Destroy()` ; : Une fois que `Run()` est terminé, cette fonction est appelée pour libérer toute la mémoire allouée aux objets de la simulation (nœuds, interfaces, applications). Cela évite les fuites de mémoire, ce qui est une bonne pratique en programmation C++.
- `return 0;` ; : Indique au système d'exploitation que le programme s'est terminé avec succès.

7 Mise en œuvre de l'Animation avec NetAnim

Pour transformer une simulation textuelle en animation graphique, NS-3 utilise le module **NetAnim**. Contrairement aux traces classiques (`.tr`), NetAnim nécessite un fichier au format **XML**.

7.1 Configuration du script C++

Trois étapes sont nécessaires dans le code source pour générer l'animation :

1. **Inclusion du module** : Ajouter l'en-tête spécifique en haut du fichier.

```
1 #include "ns3/net-anim-module.h"
2
```

2. **Instanciation de l'interface** : Créer l'objet `AnimationInterface` avant l'instruction `Simulator::Run()`.

```
1 AnimationInterface anim ("simulation_tp1.xml");
2
```

3. **Personnalisation des nœuds (Optionnel)** : Définir des noms ou des positions pour faciliter la visualisation.

```
1 anim.UpdateNodeDescription(nodes.Get(0), "Client");
2 anim.UpdateNodeDescription(nodes.Get(1), "Serveur");
3 anim.SetConstantPosition(nodes.Get(0), 10.0, 30.0);
4 anim.SetConstantPosition(nodes.Get(1), 50.0, 30.0);
5
```

7.2 Procédure de visualisation

Une fois le code compilé et exécuté via le terminal (`./ns3 run...`), la procédure suivante doit être appliquée :

- **Fichier XML** : Le simulateur génère un fichier `simulationtp1.xml` à la racine du dossier NS-3. *Note : Le fichier `.tr` n'est pas utilisé par NetAnim.*
- **Lancement de NetAnim** : Exécuter le logiciel via le terminal dans son dossier respectif (`./NetAnim`).
- **Chargement** : Cliquer sur l'icône de dossier (Open) dans l'interface graphique et sélectionner le fichier `.xml`.

- **Lecture** : Utiliser le bouton *Play* pour visualiser les flux de paquets entre les nœuds.

Note importante : Le fichier **.xml** contient les positions des nœuds et les événements de paquets. Si aucune position n'est définie dans le code C++, tous les nœuds apparaîtront superposés au point (0,0) dans l'animateur.

8 Résumé Objectifs du TP1

Le script `first.cc` permet de comprendre les quatre piliers d'une simulation NS-3 :

1. La création de la topologie (*Nodes* et *Devices*).
2. L'installation de la pile réseau (*Internet Stack* et *IP*).
3. La configuration des applications (*Traffic Generators*).
4. La gestion du cycle de vie (*Run* et *Destroy*).

Application : Analyse du script

first.cc

Travaux Pratiques NS-3 –

Objectif

L'objectif de cet exercice est de vérifier votre compréhension de la structure d'une simulation réseau et de l'interaction entre les différents *Helpers* de NS-3.

Consigne : En vous basant sur le code source étudié, répondez aux questions suivantes avec précision.

Partie 1 : Architecture de la Topologie

1. **Dimensionnement :** Combien de nœuds physiques sont créés dans cette simulation ? Identifiez la méthode C++ utilisée pour cette création.
2. **Canal de communication :**
 - Quel est le débit binaire (*Data Rate*) configuré pour la liaison ?
 - Quel est le délai de propagation (*Delay*) du canal ?
3. **Conteneurs :** Expliquez la différence de rôle entre le `NodeContainer` et le `NetDeviceContainer`. Pourquoi le second est-il le résultat de l'installation du premier ?

Partie 2 : Adressage et Flux de données

1. **Plan d'adressage :** Le réseau de base est 10.1.1.0. En sachant que NS-3 attribue les adresses de manière séquentielle, quelle sera l'adresse IP exacte du nœud serveur (n1) ?
2. **Chronologie des événements :**
 - À quel instant (*timestamp*) le serveur commence-t-il à écouter ?
 - À quel instant le client envoie-t-il son premier paquet ? Pourquoi prévoit-on un décalage entre les deux ?
3. **Charge utile :** Quelle est la taille (en octets) du paquet envoyé par le client ? Quelle méthode permet de modifier cette valeur ?

Partie 3 : Diagnostic et Logs

1. **Observation :** Quelle commande de terminal permet d'activer la journalisation (*Logging*) pour voir l'échange de paquets UDP en temps réel lors de l'exécution ?

Partie 4 : Visualisation et Animation

1. **Génération du fichier XML :** Quelle ligne de code (utilisant `AnimationInterface`) doit être ajoutée au script pour générer le fichier nécessaire à la visualisation sous NetAnim ?

2. **Positionnement des nœuds :** Par défaut, les nœuds dans `first.cc` n'ont pas de coordonnées spatiales. Quelle famille de *Helpers* permet de définir une position (x, y) ou une mobilité pour que les nœuds apparaissent distinctement dans l'interface graphique ?
3. **Analyse visuelle :** Lors de l'exécution dans NetAnim, comment sont représentés graphiquement les paquets circulant entre `n0` et `n1` ? Que permet de vérifier cette visualisation par rapport aux logs du terminal ?

Dr. KHAOUA BEN ALI

Note : Une attention particulière sera portée à la précision des termes techniques utilisés (ex : Helper, Stack, Attribute).